Identyfikacja Procesów

Dynamiczne modele liniowe obiektów niestacjonarnych

Dawid Najda, Michał Kaczmarczyk

20 grudnia 2022



Politechnika Śląska

Contents

1 Zadanie 1 - symulacja obiektu niestacjonarnego **3**

2 Zadanie 2 - identyfikacja **5**

3 Kody **8**

3.1 Zmienny filtr 8

3.1.1 Implementacja 8

3.1.2 Wywołanie 9

3.2 Estymacja parametrów 10

3.2.1 Wywołanie 10

3.2.2 Implementacja tworzenia macierzy danych 10

3.2.3 Implementacja LMS 11

3.2.4 Implementacja Kaczmarza 11

3.2.5 Implementacja RLS 12

3.2.6 Implementacja WRLS 13

1 Zadanie 1 - symulacja obiektu niestacjonarnego

Symulowany model przyjmuje postać jak na równaniu 1.

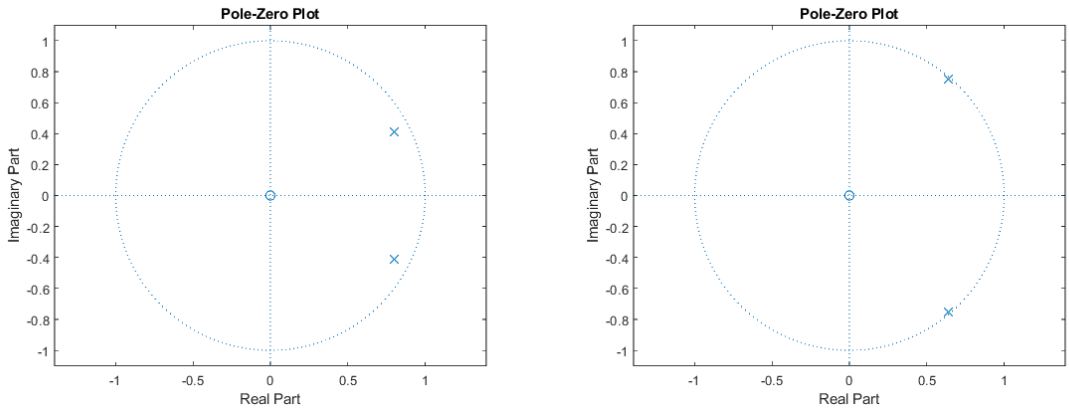
$$y(i) = z^{-1}\frac{0.7}{1 - 1.6z^{-1} + 0.81z^{-2}}u(i) \tag{1}$$

Dla tak przygotowanego modelu wyznaczono równanie różnicowe o postaci 2.

$$y(i) = 0.7u(i-1) + 1.6y(i-1) - 0.85y(i-2) \tag{2}$$

Równanie różnicowe 2 zaimplementowano w programie Matlab i pobudzono, generując sygnał wejściowy u(i) funkcją randn dla 1500 próbek. Następnie ponownie zasymulowano obiekt zmieniając parametry modelu do postaci z równania 3 od próbki 501 do 1000. Porównanie stabilności transmitancji 2 i 3 przedstawia rysunek 1. Porównanie wyjść modelu stacjonarnego i niestacjonarnego przedstawia rysunek 2.

$$y(i) = z^{-1}\frac{0.7}{1 - 1.28z^{-1} + 0.972z^{-2}}u(i) \tag{3}$$



(a) Transmitancja pierwsza

(b) Transmitancja druga

Figure 1: Zera i bieguny transmitancji

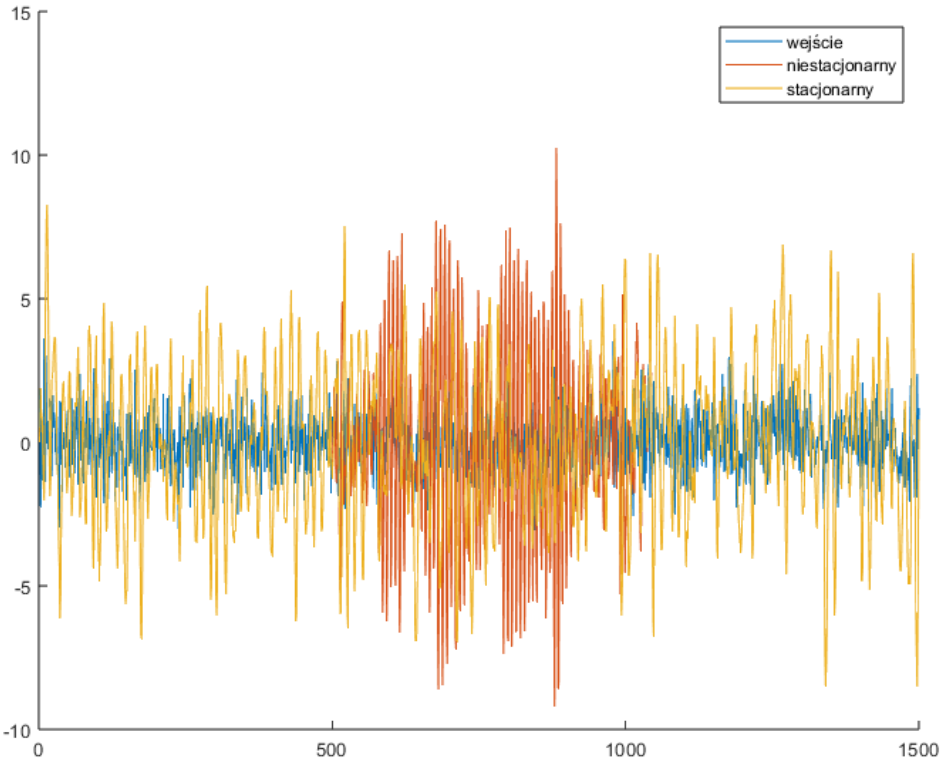


Figure 2: Przebieg czasowy wyjść obu modeli

2 Zadanie 2 - identyfikacja

Identyfikacja modelu niestacjonarnego z zadania 1 została przeprowadzona za pomocą wielu różnych algorytmów: LMS, Kaczmarza, RLS, WRLS. Przebiegi szacowań parametrów zależnie od próbki i zostały przedstawione na wykresach poniżej.

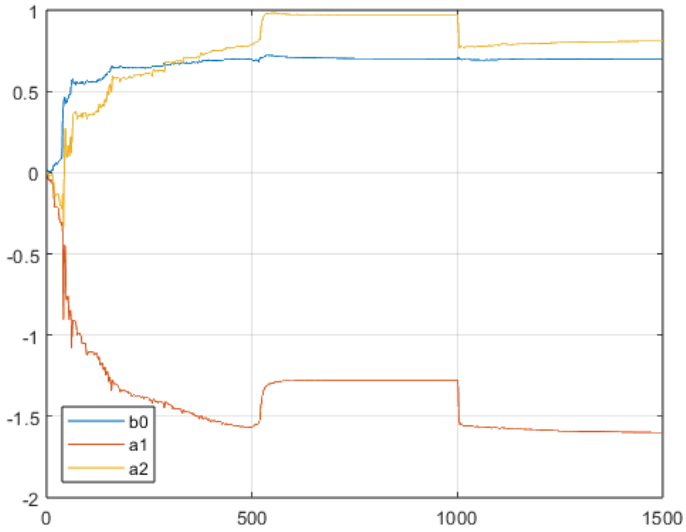


Figure 3: Algorytm LMS

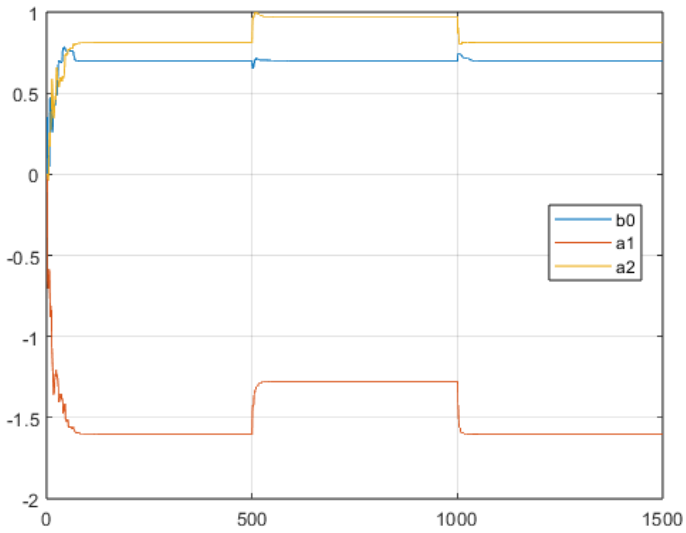


Figure 4: Algorytm Kaczmarza

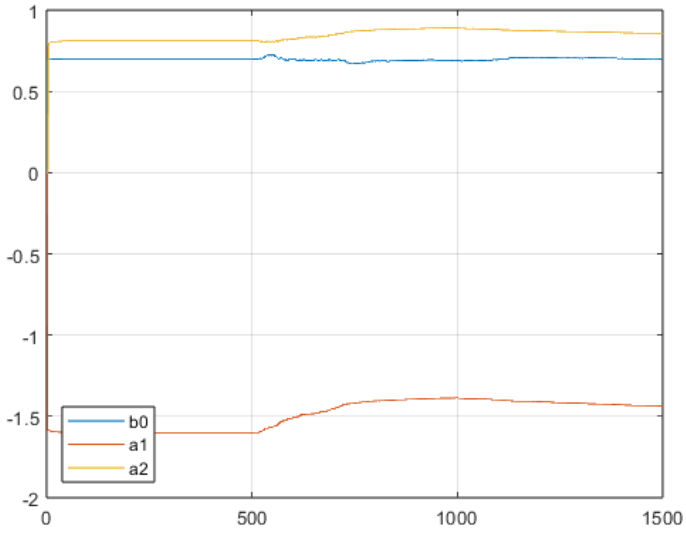


Figure 5: Algorytm RLS (1)

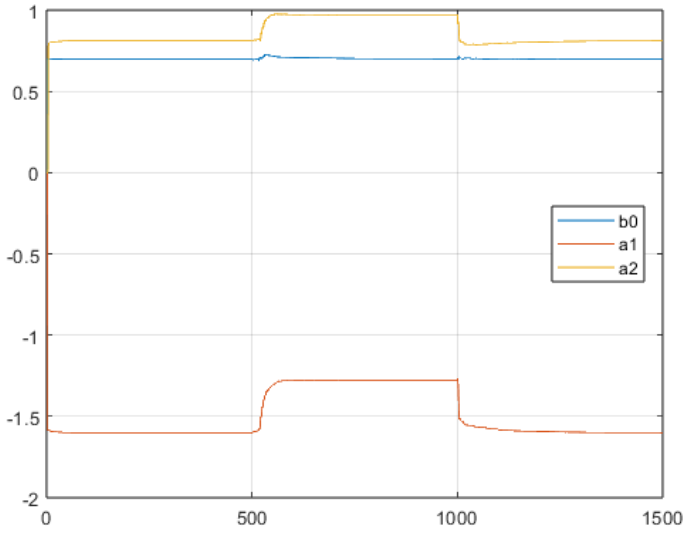


Figure 6: Algorytm RLS (2)

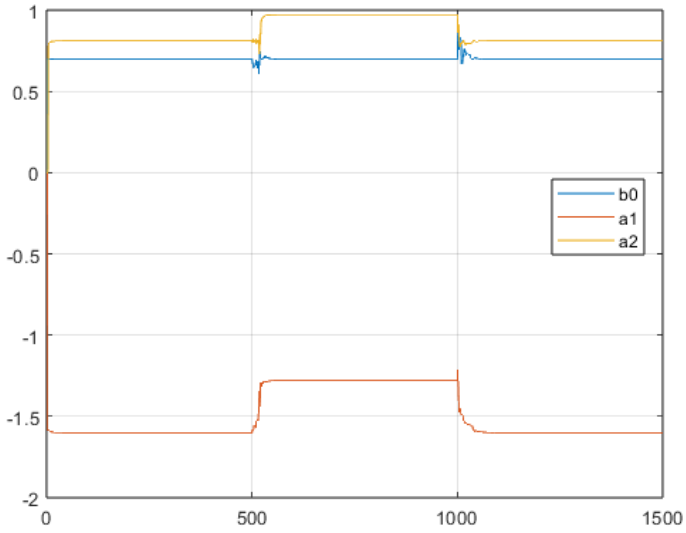


Figure 7: Algorytm WRLS

Wszystkie algorytmy spełniły swoją rolę, jedne trochę lepiej, inne trochę gorzej. Najprostszy LMS potrzebuje trochę aby się ustabilizować, jednak potem dość dobrze podąża za zmianami, czasami z opóźnieniem. Kaczmarz dociera do finalnych wartości sporo szybciej, oraz reaguje bez opóźnienia, jednak miejscami nerwowo. RLS działa dobrze ze statycznymi modelami, jednak przez bardzo mocne zredukowanie macierzy P nie jest w stanie podążać za zmianami. Druga wersja, z dolnym ograniczeniem na przekątną P, jest minimalnie mniej stabilna, jednak w zamian reaguje na zmiany, dość gładko, choć z lekkim opóźnieniem. WRLS działa podobnie; reaguje i stabilizuje się trochę szybciej, za to miejscami jest zakłócony przy zmianach.

3 Kody

3.1 Zmienny filtr

3.1.1 Implementacja

Funkcja symulująca model o zmiennych parametrach. Argumenty: macierz parametrów licznika, macierz parametrów mianownika, opóźnienie, sygnał wejściowy. Liczy w pętli kolejne wyjścia bazując na zadanych parametrach, podanych wejściach oraz poprzednich wyjściach.

```
function Y = zmiennyFiltr(B, A, k, U)

    N = length(U);
    Y = zeros(size(U));

    dA = width(A)-1;
    dB = width(B)-1;

    for i = 1:N
        y = 0;
        for j = 0:dB
            y = y + B(i, j+1) * getU(i-j-k);
        end
        for j = 1:dA
            y = y - A(i, j+1) * getY(i-j);
        end

        Y(i) = y;
    end

    function u = getU(i)
        if i > 0 && i <= N
            u = U(i);
        else
            u = 0;
        end
    end
    function y = getY(i)
        if i > 0 && i <= N
            y = Y(i);
        else
            y = 0;
        end
    end
end
```

3.1.2 Wywołanie

```
N = 1500;
n = 500;

U = randn(N, 1);

b  = [0.7];
a  = [1 -1.6     0.81    ];
am = [1 -1.6*0.8 0.81*1.2];

k = 1;

B =  repmat(b, N, 1);
A = [repmat(a, n, 1); repmat(am, n, 1); repmat(a, n, 1)];

Y = zmiennyFiltr(B, A, k, U);

figure
plot(1:N, Y);
```

3.2 Estymacja parametrów

3.2.1 Wywołanie

```
load('UY.mat');

ests = LMS(U, Y, 0, 2, 1, 0.01);

ests = Kaczmarz(U, Y, 0, 2, 1, 0.5);

ests = RLS(U, Y, 0, 2, 1, 100, 0); % brak podążania za zmianami
ests = RLS(U, Y, 0, 2, 1, 100, 0.01);

ests = WRLS(U, Y, 0, 2, 1, 100, 0, 0.9);

figure
plot(ests)
grid on
legend('b0', 'a1', 'a2', 'Location','southwest')
```

3.2.2 Implementacja tworzenia macierzy danych

Funkcja tworzy macierz danych. Argumenty: sygnał wejściowy, sygnał wyjściowy, rząd licznika, rząd mianownika, opóźnienie. Tworzy odpowiednią ilość kolumn z odpowiednimi przesunięciami indeksów.

```
function DATA = getUmatrix(U, Y, dB, dA, k)
    N = length(Y);
    d = dB + 1 + dA;

    DATA = zeros(N, d);

    j = 1;
    for i = 1:dB+1
        DATA( (i+k):end , j) = U( 1:(end-i-k+1) );
        j = j+1;
    end
    for i = 2:dA+1
        DATA( i:end , j) = -Y( 1:(end-i+1) );
        j = j+1;
    end
end
```

3.2.3 Implementacja LMS

Funkcja implementuje algorytm LMS. Argumenty: sygnał wejściowy, sygnał wyjściowy, rząd licznika, rząd mianownika, opóźnienie, krok.

```
function estymators = LMS(U, Y, dB, dA, k, mu)

    N = length(Y);
    d = dB + 1 + dA;

    DATA = getUmatrix(U, Y, dB, dA, k);

    estymators = zeros(N, d);
    est = zeros(1, d);

    for i = (k+1):N
        u = DATA(i, :);
        err = Y(i) - est*u';
        est = est + mu * u * err;
        estymators(i, :) = est;
    end
end
```

3.2.4 Implementacja Kaczmarza

Funkcja implementuje algorytm LMS. Argumenty: sygnał wejściowy, sygnał wyjściowy, rząd licznika, rząd mianownika, opóźnienie, krok.

```
function estymators = Kaczmarz(U, Y, dB, dA, k, mu)

    N = length(Y);
    d = dB + 1 + dA;

    DATA = getUmatrix(U, Y, dB, dA, k);

    estymators = zeros(N, d);
    est = zeros(1, d);

    for i = (k+1):N
        u = DATA(i, :);
        err = Y(i) - est*u';
        est = est + mu * u/(u*u') * err;
        estymators(i, :) = est;
    end
end
```

3.2.5 Implementacja RLS

Funkcja implementuje algorytm RLS. Argumenty: sygnał wejściowy, sygnał wyjściowy, rząd licznika, rząd mianownika, opóźnienie, maksymalna wartość na przekątnej P, minimalna wartość na przekątnej P.

```
function estymators = RLS(U, Y, dB, dA, k, lmax, lmin)

    N = length(Y);
    d = dB + 1 + dA;

    DATA = getUmatrix(U, Y, dB, dA, k);

    Pmax = Inf(d);
    Pmin = -Inf(d);
    Pmax(1:(d+1):end) = lmax;
    Pmin(1:(d+1):end) = lmin;
    P = eye(d)*lmax;

    estymators = zeros(N, d);
    est = zeros(1, d);

    for i = (k+1):N
        u = DATA(i, :);
        P = P - P*(u'*u)*P / (1 + u*P*u');
        P = min(Pmax, max(Pmin, P));
        k = P * u';

        err = Y(i) - est*u';
        est = est + k' * err;
        estymators(i, :) = est;
    end
end
```

3.2.6 Implementacja WRLS

Funkcja implementuje algorytm RLS. Argumenty: sygnał wejściowy, sygnał wyjściowy, rząd licznika, rząd mianownika, opóźnienie, maksymalna wartość na przekątnej P, minimalna wartość na przekątnej P, współczynnik zapominania.

```
function estymators = WRLS(U, Y, dB, dA, k, lmax, lmin, alpha)

    N = length(Y);
    d = dB + 1 + dA;

    DATA = getUmatrix(U, Y, dB, dA, k);

    Pmax = Inf(d);
    Pmin = -Inf(d);
    Pmax(1:(d+1):end) = lmax;
    Pmin(1:(d+1):end) = lmin;
    P = eye(d)*lmax;

    estymators = zeros(N, d);
    est = zeros(1, d);

    for i = (k+1):N
        u = DATA(i, :);
        P = ( P - P*(u'*u)*P / (alpha + u*P*u') ) / alpha;
        P = min(Pmax, max(Pmin, P));
        k = P * u';

        err = Y(i) - est*u';
        est = est + k' * err;
        estymators(i, :) = est;
    end
end
```

List of Figures

1 Zera i bieguny transmitancji 3
2 Przebieg czasowy wyjść obu modeli 4
3 Algorytm LMS 5
4 Algorytm Kaczmarza 5
5 Algorytm RLS (1) 6
6 Algorytm RLS (2) 6
7 Algorytm WRLS 7